# The CPU: A very tall pile of simple

Part 3: Execution

*Ekansh Goenka*

October 2026

# Contents

**3.1. Instructions Are Numbers ..... 3**
**3.2. The Not-So-Clever Control Unit ..... 6**
**3.3. Finally, The Fun Part ..... 19**
**3.4. The End ..... 22**

# 3.1. Instructions Are Numbers

---

Instructions tell the CPU what to do, and a list of instructions is a program. There are game programs, calculator programs, email programs. Everything on your computer is a program. Just a long, long list of instructions telling the CPU to do simple things like add these two numbers, move this value here, etc.

All programs are just a list of instructions stored in `RAM`, each instruction coded as a number.

So here is what number each instruction in our CPU translates to:

| # | Binary | Mnemonic | Bytes | Action |
|---|---|---|---|---|
| 0 | `0000` | `NOP` | 1 | do nothing |
| 1 | `0001` | `LOAD A, n` | 2 | next byte -> A |
| 2 | `0010` | `LOAD A, [addr]` | 2 | RAM slot -> A |
| 3 | `0011` | `STORE A, [addr]` | 2 | A -> RAM slot |
| 4 | `0100` | `MOV B, A` | 1 | copy A into B |
| 5 | `0101` | `MOV A, B` | 1 | copy B into A |
| 6 | `0110` | `JMP addr` | 2 | addr -> PC |
| 7 | `0111` | `JE addr` | 2 | if `EQUAL` flag: addr -> PC |
| 8 | `1000` | `ADD` | 1 | A = A + B |
| 9 | `1001` | `AND` | 1 | A = A AND B |
| 10 | `1010` | `OR` | 1 | A = A OR B |
| 11 | `1011` | `XOR` | 1 | A = A XOR B |
| 12 | `1100` | `JC addr` | 2 | if `CARRY` flag: addr -> PC |
| 13 | `1101` | `JZ addr` | 2 | if `ZERO` flag: addr -> PC |
| 14 | `1110` | `HALT` | 1 | stop the clock |

*Table 12.1. The full instruction set.*

So we store `00000001` in address 0 of `RAM`. Then store `00000101` in address 1 of `RAM`. Then store `00000011` in address 2, then `11111111` in address 3, and finally `00001110` in address 4.

Our `RAM` looks like this:

```
0: 00000001
1: 00000101
2: 00000011
3: 11111111
4: 00001110
```

If `PC` starts at 0, which it does, then what do you think this program would do?

The first instruction loads a number into register `A`. That number is 5, and we know that because the value stored in address 1 of `RAM` is binary for 5, so those two bytes make up the instruction:

`LOAD A, 5`

The first byte tells us what the instruction is; the second tells us what value to load. The next instruction is `3` which is `STORE A, [addr]` which basically puts the contents of `A` into the address of `RAM` that we specify. Again, the first byte tells us the instruction and the second byte tells us the address, in this case `11111111`, which is address 255.

Hey? Isn't address 255 different from the rest? Yep, if we write to address 255 we also write to the `DISPLAY` register where we can see that value on some light bulbs. Have a look at this diagram for a recap.

The last byte is `14`, which the table says is `HALT`. It stops the clock, the whole machine freezes, and the program is over.

But, why do we even need to stop the CPU? Well, if we don't, `PC` would just keep counting up and the CPU would keep fetching. It doesn't know our program ended at address 4. It would march straight into the rest of `RAM` and run whatever is sitting there as if it were instructions, leftovers from the last program, or whatever junk the memory happened to wake up holding. We need a way to stop.

So this program just puts 5 on the output and stops. Pretty simple.

As you can tell, some instructions take two bytes, and some only need one. A 2-byte instruction is one where the first byte alone is not enough. `LOAD A, n` needs to know what value to load, so the next byte in `RAM` is that value.

One more thing: the order things are written in. For `MOV`, the destination comes first. `MOV B, A` copies `A` into `B`, not the other way around. `LOAD` and `STORE` are different: `LOAD A` means a value going into `A`, and `STORE A` means `A` going out into memory.

Also, a number with brackets like `[5]` means whatever is stored at address 5, but just `5` means literally the number 5, which would be stored in the next byte of `RAM`. It's just an easier way for us humans to write it.

Now, about the jump instructions: `JMP`, `JE`, `JC`, and `JZ`. All of these instructions set `PC` to the address stored in the second byte.

`JE`, `JC`, and `JZ` each check one flag. The flags from the last `ALU` instruction sit in the `FLAGS` register until the next `ALU` instruction overwrites them. So to make a decision, you run an `ALU` instruction, then jump based on what it found. If the flag is on, the jump sets `PC` to that address. If not, the program just carries on to the next instruction. This is the core element used in creating programs that can decide stuff and loop. A fundamental part of what a computer can do relates to being able to run in a loop.

For example:

```
0: LOAD A, 1
2: STORE A, [255]
4: MOV B, A
5: ADD
6: JMP 2
```

This jumping back to the start of the loop is done with a jump instruction, so for example whenever `PC` reaches 6, it jumps back to 2 and repeats all over again. Each loop, this program displays `A`, copies `A` into `B`, and then adds. Both `ALU` inputs are the same number, so `A` doubles

every loop, and the output shows 1, 2, 4, 8, 16, all the way up to 128. On the 8 output bulbs, that looks like a single lit bulb going across the display. Pretty neat.

But then 128 + 128 overflows: `A` wraps around to 0. And from that point on, the program is doubling 0 forever, so the display goes dark and stays dark. The CPU gets stuck in a weird spot.

One note: the number before each instruction is just the address in `RAM` where it would be stored, that is why you can see it jump in twos for 2-byte instructions.

To solve this overflow problem, and in general to make decisions, we can use the flags to jump to different parts of our program depending on what the last `ALU` instruction found.

```
0: LOAD A, 1
2: STORE A, [255]
4: MOV B, A
5: ADD
6: JC 0
8: JMP 2
```

Now, when the addition overflows, the `CARRY` flag turns on, and `JC` catches it: we jump back to the start of the program, address 0, which loads 1, and the program starts all over again. Forever. On every normal loop, `JC` checks the flag, finds it off, and the program just carries on to the `JMP`.

Because this computer is built from relays, and thus pretty slow, we can probably watch the light walk across the display with our own eyes. If we want to slow it down some, we can pad the program with some `NOP`s that waste clock cycles.

Real computers are insanely fast and usually have specialized timer hardware and such, but sometimes just run "do-nothing loops" that just waste clock cycles for a set time. Here is an example of a simple do-nothing delay loop:

```
0:  LOAD A, 1
2:  MOV B, A
3:  LOAD A, 0
5:  ADD
6:  JC 10
8:  JMP 5
10: (rest of the program)
```

All this loop does is add 1 to `A` over and over, 256 times, until the addition overflows and the `CARRY` flag lets it escape. It computes nothing useful, it just eats up time.

But we don't really need these for our slow computer. Just know real computers use a combination of timer hardware, little counter circuits that tick along with the clock, and loops like these to wait for the right amount of time. Some CPUs even have a sleep instruction that shuts them down completely until something wakes them up.

So basically, these jump instructions can help us branch on certain conditions letting our program decide stuff, and also run in loops.

But how would the CPU actually execute an instruction? What control wires would it flip, and in what order? Let's take the `MOV B, A` instruction as an example.

First the `CU` would flip the `O` signal on the `A` register, driving the bus to whatever is stored in `A`. Then the `CU` would flip the `W` signal on register `B`, writing the value on the bus into the register. Then it would turn off the `O` signal on register `A`, and boom, we are done. The `CU` had to take a few tiny micro-steps to execute that instruction. We will talk about micro-steps in more detail later on.

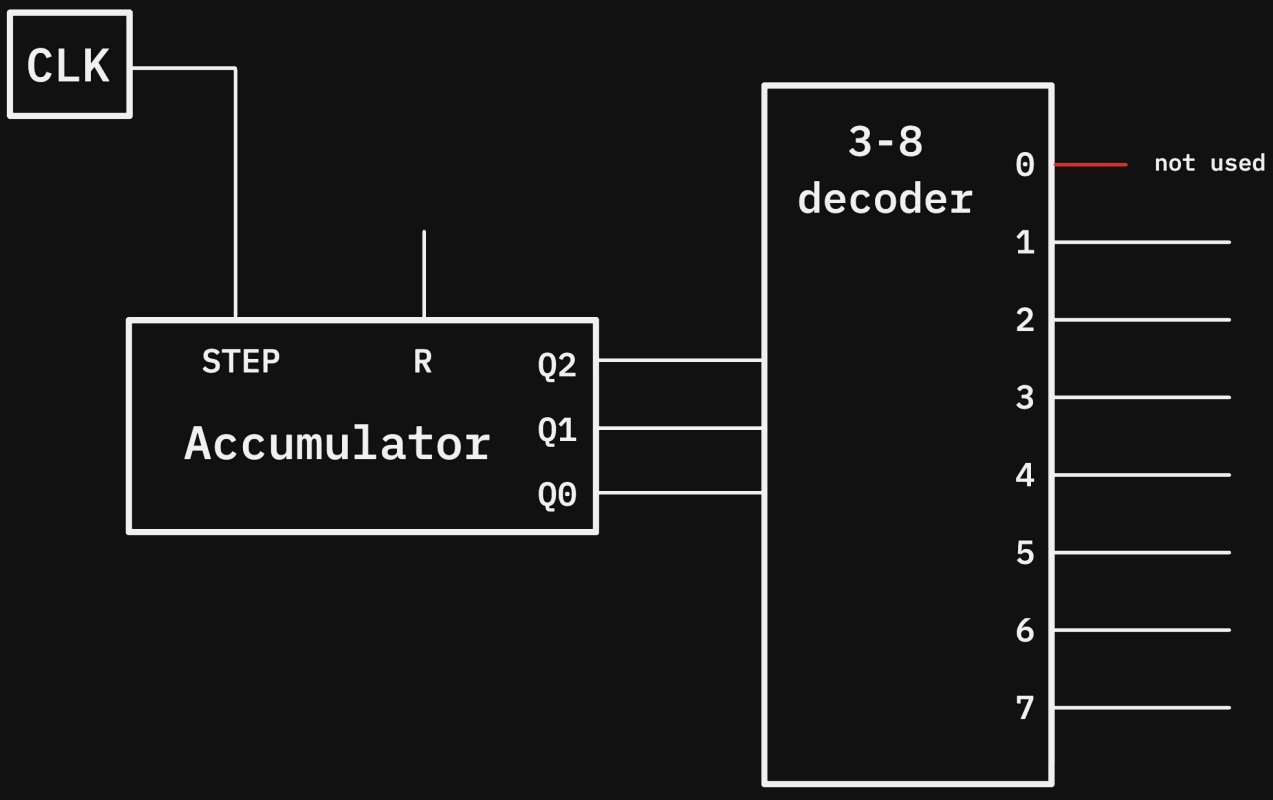
How does the `CU` know which wires to flip and when? That is what the next section is all about.

## 3.2. The Not-So-Clever Control Unit

---

The first thing the control unit needs is a step counter. Something that counts, step 0, then step 1, then step 2, and so on, back to step 0. That is the core flow of the control unit.

We can combine a simple accumulator and a `3-8` decoder. This will give us 8 steps, and because the accumulator overflows and naturally wraps back around to 0, the steps will loop correctly.




Full 16-frame animation at bytecolony.computer/writing/the-cpu/#f=cu-counter:1.

**Diagram 13.1.** The step counter.

The accumulator part of the step counter is connected to the clock. The clock is the heartbeat of the computer, and it lives right here in the control unit.

One thing: remember the TAKEOVER switch? Well when it is on it basically resets this step counter's accumulator to 0, and on 0 nothing happens. So every instruction starts at step 1, and step 0 is just unused. This also means `TAKEOVER` is a reset switch. Flick it on and off, and both `PC` and the stepper get reset to 0.

So basically each of these wires turning on is 1 micro-step. If we want to execute an instruction like `MOV B, A` we need to enable the output of `A` onto the bus, and then enable `W` on `B` to store that value.

If we tried to just turn `O` and `W` on at the same time, the register might capture the value on the bus before the other register has started driving the bus. The bus hasn't had time to settle. So this won't work. What we want is to turn `O` on, wait for the bus to settle, and then turn `W` on, right in the middle of `O` being on.

We could try spreading this across two steps: turn `O` on for step 1, then keep `O` on for step 2 and turn on `W` too. The problem is that when the counter switches from one step to the next, the decoder briefly flickers through the wrong output, so `O` can drop at the exact moment `W` fires. And every move would now cost two steps instead of one. So yea, it's just not a good way to do it.

The best approach to this is just taking our clock signal and flipping it:

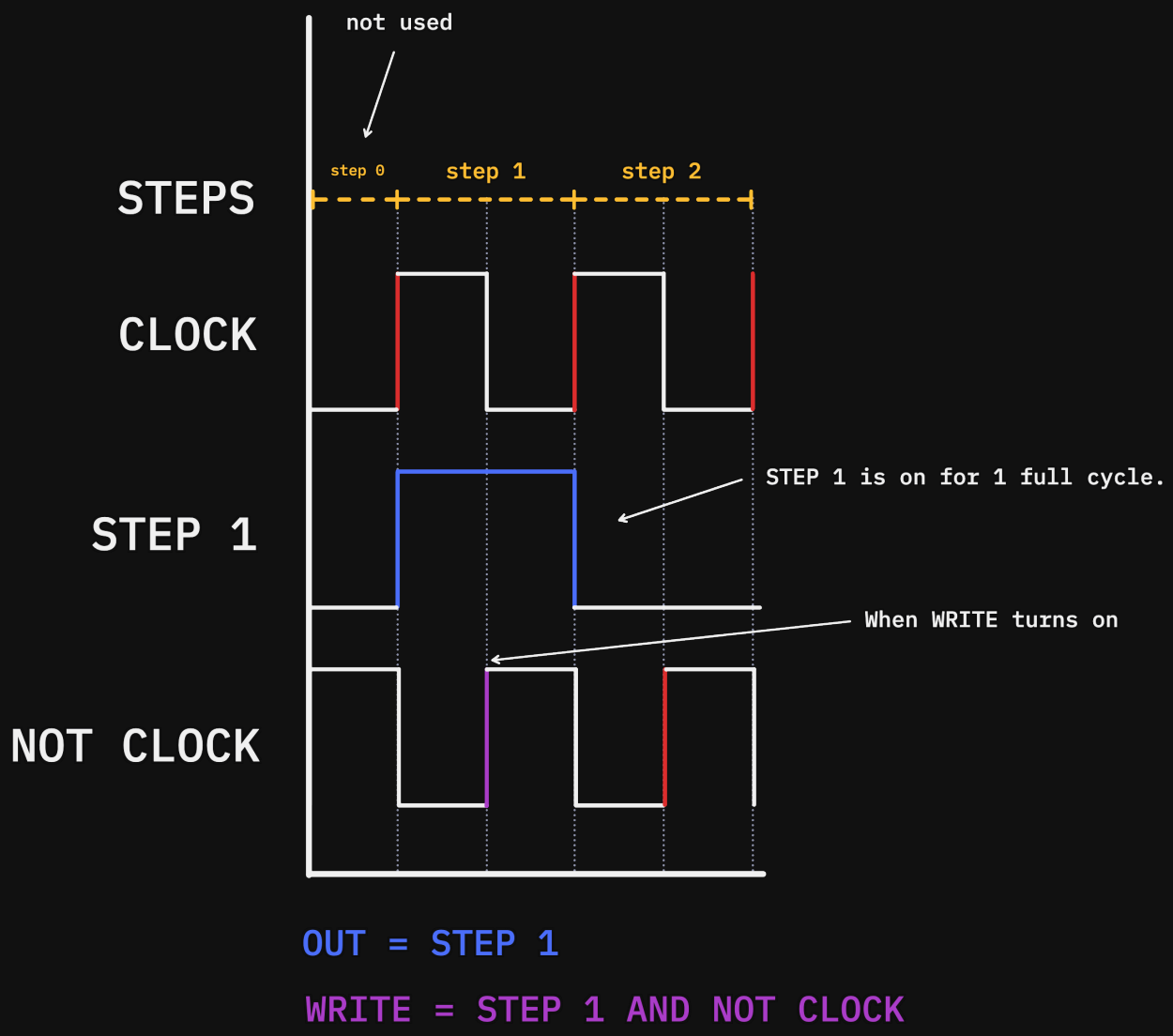



**Diagram 13.2.** Micro-step timing diagram.

Purple marks when `W` turns on, and blue is when `O` is on.

Each step changes on a clock edge, and each step is a signal, 1 full clock cycle long, until the next edge.

`O` is directly connected to that step line. If we enable `O` on step 1, the bus is driven for that full cycle.

`W` is ANDed with `NOT CLOCK`, like `step 1 AND NOT CLOCK`. The thing is, step and `NOT CLOCK` don't come on at the same time. The step line is already on, and then halfway through the cycle `NOT CLOCK` goes up, and that is when the AND gate's output goes up. That is the edge `W` needs, right in the middle, after `O` has had some time to settle.

Now that we have covered that, let's figure out what should happen on each step.

The first thing the `CU` will have to do for every instruction no matter what is to actually fetch that instruction from `RAM` and store it in `IR`. `IR` stands simply for `Instruction Register` and is used to store the instruction so the `CU` can decode it and figure out what it means.

The register `PC` stands for `Program Counter` and all it does is store the address of the next byte that we need to fetch. Then after we have fetched that instruction it usually gets incremented by 1, but with jump instructions, the value in `PC` gets changed, so the program starts executing from somewhere else.

To fetch the next instruction, first we would need to put the value in `PC` into the `MAR`. Then activate `RAM_OUT` putting that instruction on the common bus, and then storing that value inside the `IR` register. Then we increment `PC` so it points at the next byte in `RAM`, sometimes this next byte is an instruction and sometimes it's the second part of a 2-byte instruction.

1. `PC_OUT`, `MAR_WRITE`
2. `RAM_OUT`, `IR_WRITE`, `PC_INC`

So to accomplish this, we only need 2 simple micro-steps.

We know how we can output and write to a register in one micro-step, but three things? How can we increment `PC` too! Well incrementing `PC` has nothing to do with the bus and doesn't disrupt anything else going on, so we just can!

And notice, these 2 micro-steps are exactly the same for every single instruction.

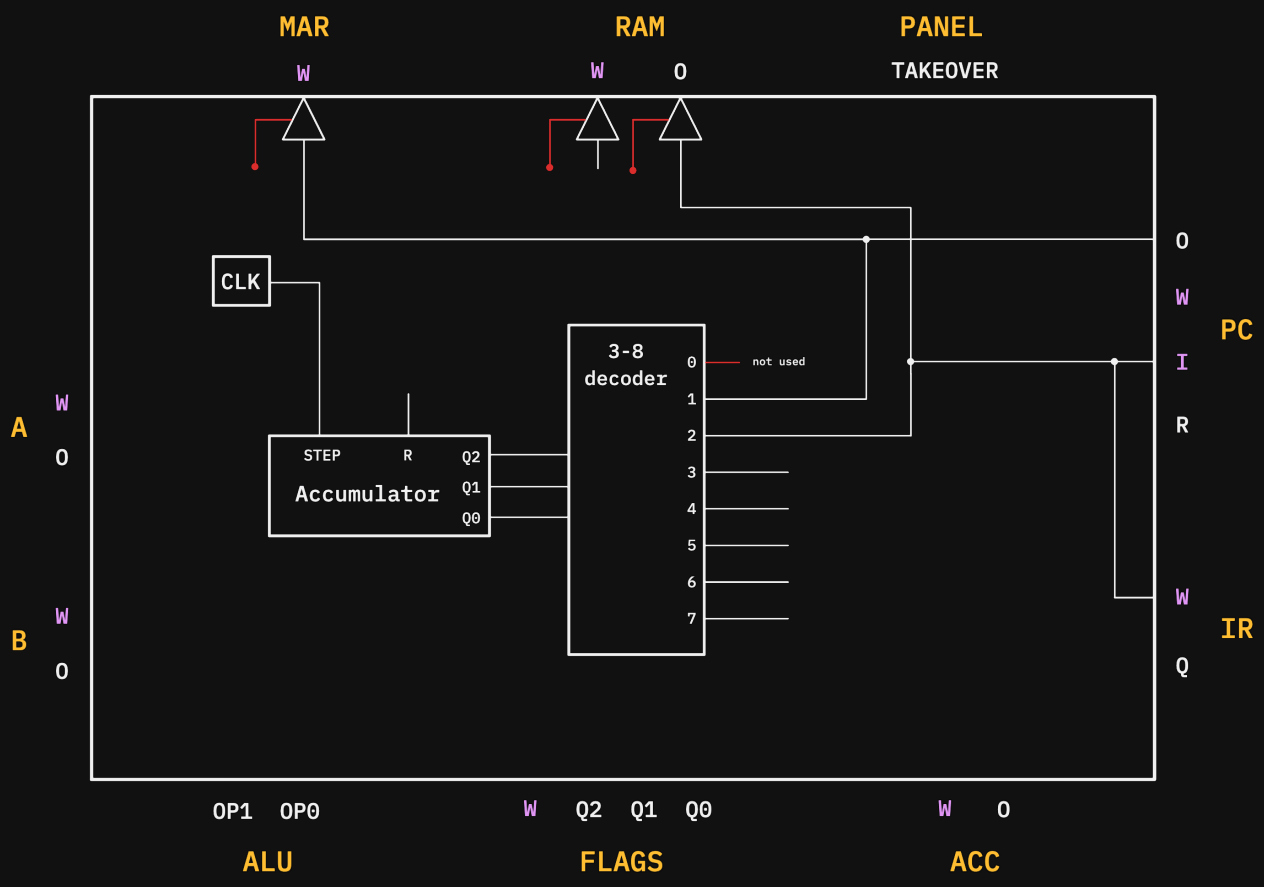



**Diagram 13.3.** The first few micro-steps.

One thing I am not drawing: every `W` signal is ANDed with `NOT CLOCK`, like we just discussed. The same goes for `PC`'s `I` signal: incrementing stores a value too, it just comes from `PC`'s own little adder instead of the bus, so it waits for `NOT CLOCK` like every other store. Drawing that same little AND gate nine times would clutter everything up, so instead, every pin colored pink has it secretly. Pink pin means stores on `NOT CLOCK`.

One more thing, I have used tri-state buffers before the `RAM` and `MAR` control signals, and I have kept the enable on for now. You can mostly ignore this for now, but it will come back later.

The rest of the micro-steps depend on the instruction we want to execute, so we use another decoder, this time a `4-16` decoder because each instruction is 4 bits long, but we only have 15 instructions (0-14) so we can ignore output 15 of the decoder, and also output 0, since `0` is a `NOP` which if we remember does nothing.

Let's first focus on the 2-byte instructions, the instructions that need to fetch a second byte. Instructions like `LOAD A, n`, `LOAD A, [addr]`, `STORE A, [addr]`, `JMP addr`, `JE addr`, `JC addr`, `JZ addr`.

Because all of these instructions need to fetch another byte, the third micro-step is always the same:

3. `PC_OUT`, `MAR_WRITE`

So we can just check, "is this instruction one of the 2-byte instructions?", using a giant OR gate, and if it is, on step 3, we can just execute the same micro-step.

The steps 4 and 5 of these instructions all slightly vary, but nothing a few logic gates can't solve.

| # | Instruction | Step 4 | Step 5 |
|---|---|---|---|
| 1 | `LOAD A, n` | `RAM_OUT`, `A_WRITE`, `PC_INC` | Nothing |
| 2 | `LOAD A, [addr]` | `RAM_OUT`, `MAR_WRITE`, `PC_INC` | `RAM_OUT`, `A_WRITE` |
| 3 | `STORE A, [addr]` | `RAM_OUT`, `MAR_WRITE`, `PC_INC` | `A_OUT`, `RAM_WRITE` |
| 6 | `JMP addr` | `RAM_OUT`, `PC_WRITE` | Nothing |
| 7 | `JE addr` | `RAM_OUT`, `PC_WRITE` if `EQUAL` else `PC_INC` | Nothing |
| 12 | `JC addr` | `RAM_OUT`, `PC_WRITE` if `CARRY` else `PC_INC` | Nothing |
| 13 | `JZ addr` | `RAM_OUT`, `PC_WRITE` if `ZERO` else `PC_INC` | Nothing |

And that "if else" in the jump rows? It is just one more AND gate and a NOT gate. If the flag is on, then we do `PC_WRITE`. If the flag is off we do `PC_INC`, or in simple terms, if we want to actually do that jump, then we need to store the new address into `PC`, if we don't actually do that jump, we just increment `PC` and move on with our lives.

Feel free to try and step through this diagram, or don't, it's a giant mess:

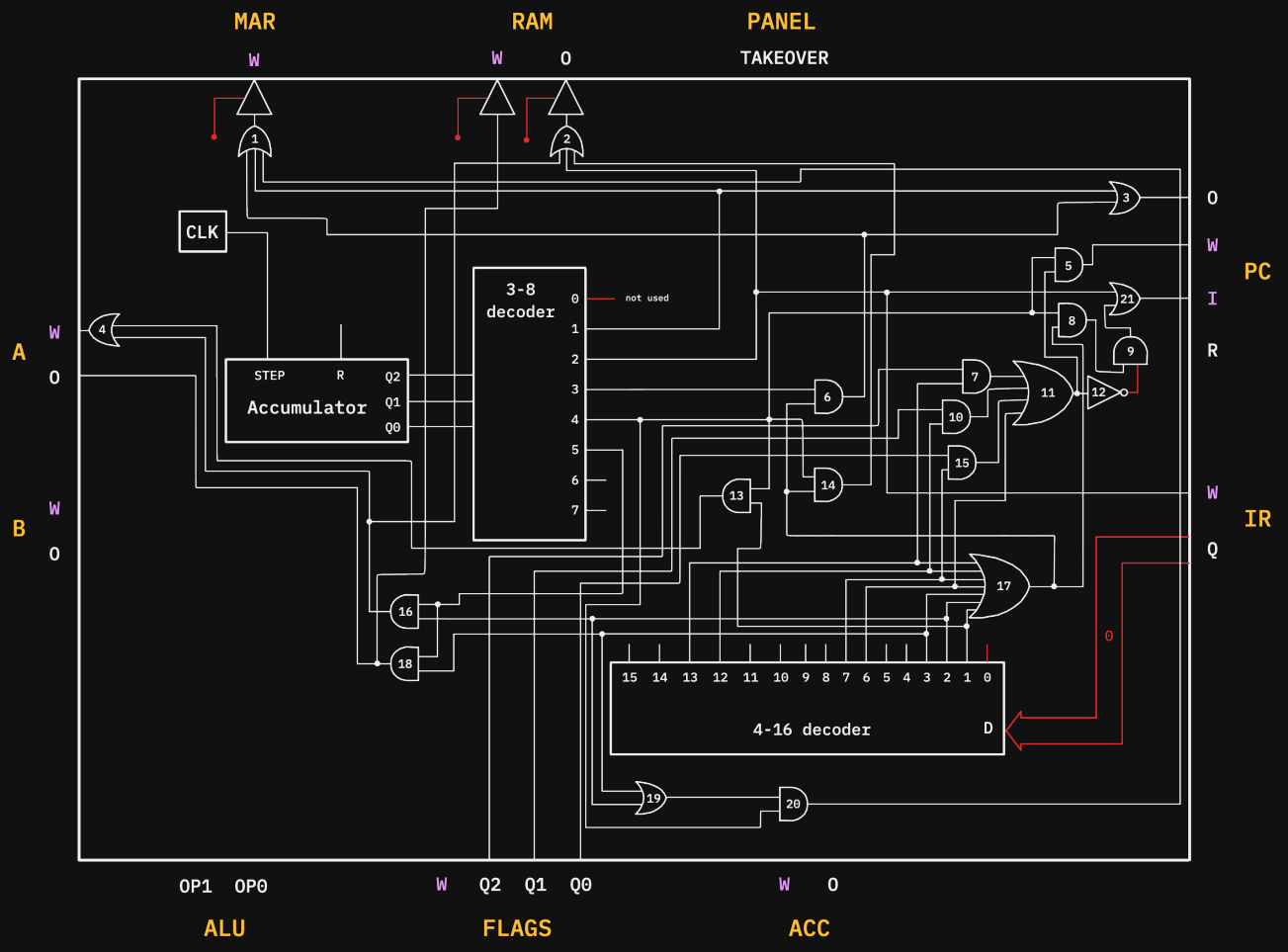



**Diagram 13.4.** All of the 2-byte instructions.

I know this is quite overwhelming, so let me break it down for you. We have the steps 1 and 2 wired just like before to fetch our instruction. Step 3 goes into gate 6, which is an AND gate. It basically checks, if it is step three, and this instruction is a 2-byte instruction. Gate 17, the OR gate basically just checks if we have a 2-byte instruction or not.

So back to step 3, if we have a 2-byte instruction and it is step three, then it enables `PC_OUT` and `MAR_WRITE`, and the `MAR_WRITE` store actually fires on `NOT CLOCK`. Step three is the same for all 2-byte instructions since we always need to fetch that second byte, but what we do with that second byte changes.

On step 4, every 2-byte instruction needs to use that second value, so we always call `RAM_OUT` if it is a 2-byte instruction. That is what gate 14 does. Then each instruction does something slightly different with that number.

Instruction 1 just stores it straight into register `A`. Gate 13 checks if it is instruction 1, and if so enables `A_WRITE`, gated of course, only actually enabling on `NOT CLOCK`.

Instructions 2 and 3 are different. For those, that second value is not the number we want, it is the *address* of the number we want. So instead of storing it, we feed it into the `MAR`. Gate 19 checks if it is instruction 2 or 3, then gate 20 checks if it is also step 4, and if so it

enables `MAR_WRITE`. Now the `MAR` is storing the address of the right slot, all ready for step 5 to do the read or write.

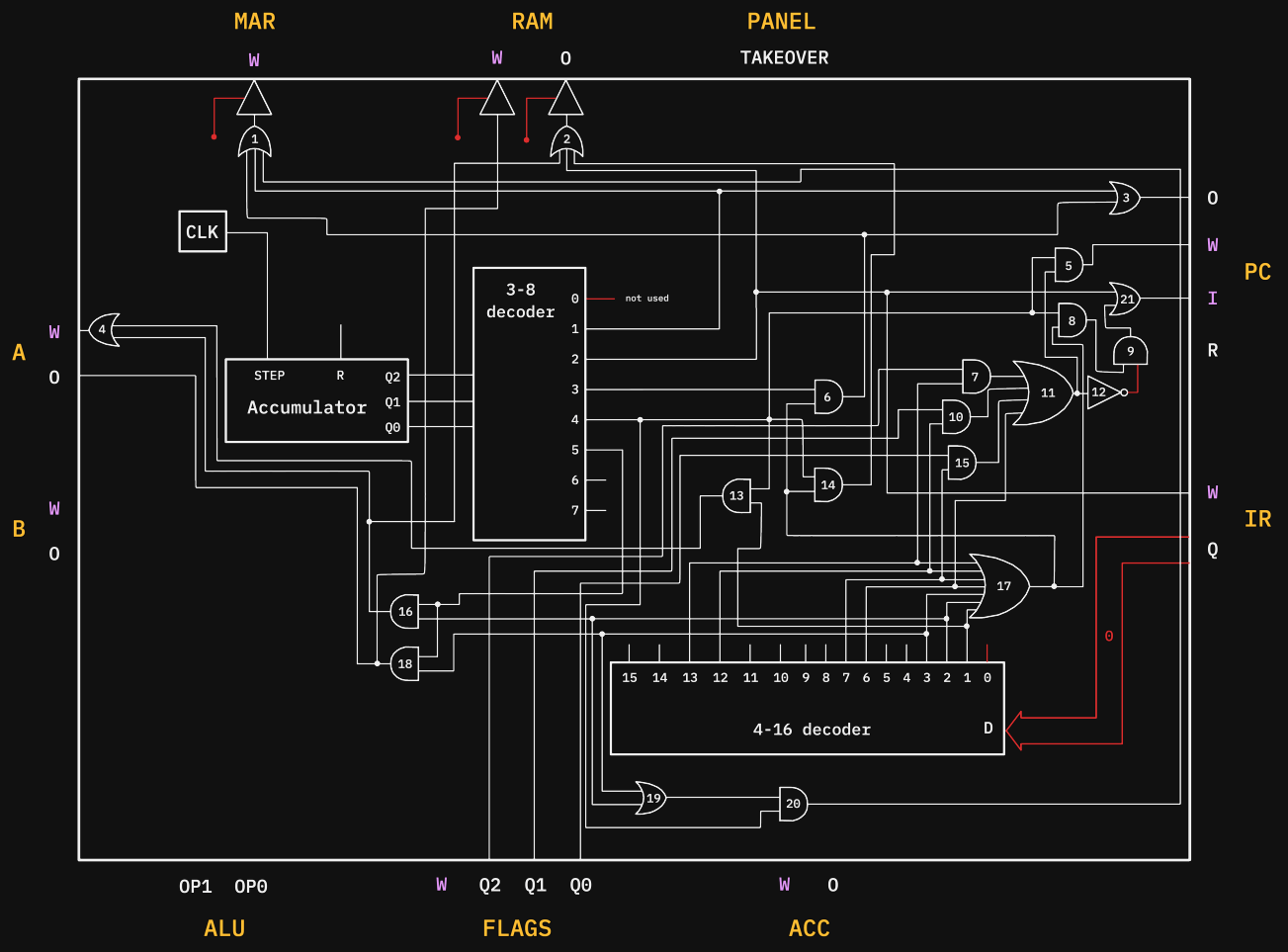
Now the jumps. For these, the second value of the instruction is a target address, so it needs to go into the `PC`. But first we have to figure out whether the jump actually happens. Gates 7, 10, and 15 do that: each one pairs a jump with its flag, so `JE` is checked against `EQUAL`, `JC` against `CARRY`, and `JZ` against `ZERO`. Those, plus a plain `JMP` which always jumps, all feed into gate 11, one big OR. If any of them fire, gate 11 turns on, meaning "this jump is happening".

From there it is simple. Gate 5 takes that signal and checks it is step 4, then enables `PC_WRITE`, loading the target address into `PC`, so then the program will start executing from that target address on the next fetch. Gate 12 then flips gate 11's output, so when the jump does not happen, it enables `PC_INC` instead, letting the program carry on to the next instruction. That is the whole "if else" we were talking about in that table. Quite straightforward.

At this point only two instructions even use step 5, `LOAD A, [addr]` and `STORE A, [addr]`. Gates 16 and 18 check if it is step 5 and either one of those instructions, then the appropriate control wires get turned on. Pretty simple.

Also, you might notice, control signals that get activated for different reasons have a "collector" OR gate instead of everything being wired together. This is just to prevent a short circuit, one wire driving low and another driving high. A simple OR gate solves this. Gates 1, 2, 3, and 4 are these collectors, feel free to check them yourself.

Here is the process of executing the instruction `LOAD A, [addr]`. We first have to fetch the instruction, then fetch the `[addr]`, then store whatever lives at that address into register `A`.




Full 16-frame animation at bytecolony.computer/writing/the-cpu/#f=cu-internals-two-demo:1.

**Diagram 13.5.** `LOAD A, [addr]` animation.

When the label for a control signal turns red, that means it's on. For the gated signals colored in pink, they will only actually turn on at `NOT CLOCK`.

Next we do the `MOV` instructions.

| # | Instruction | Step 3 |
|---|---|---|
| 4 | `MOV B, A` | `A_OUT, B_WRITE` |
| 5 | `MOV A, B` | `B_OUT, A_WRITE` |

Quite simple:

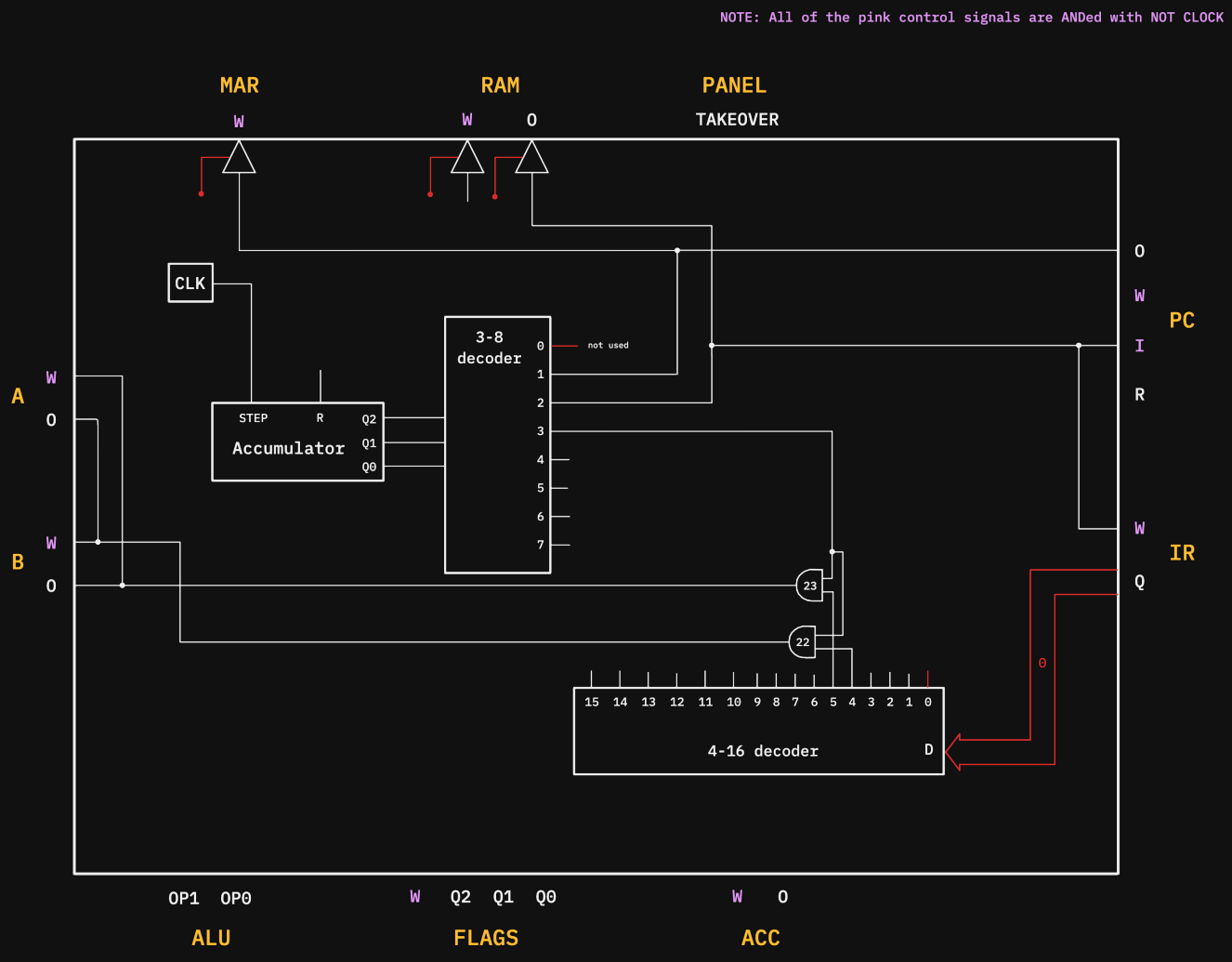



**Diagram 13.6.** The MOV instructions.

I have left out all of the 2-byte instructions from this diagram, just so it's easier to read.

Next we need to decode all of our `ALU` instructions, now this is where things get a bit weird, and I have some explaining to do.

In our CPU, if you remember, register `A` has a special `Q` that bypasses the tri-state buffers, and feeds directly into the `ALU`. So to use our `ALU`, the programmer would first put their first number into register `A` using some instructions of their choice, and put their second number into register `B`. Then any `ALU` instruction would automatically use registers `A` and `B` as the two numbers, and the `ALU` stores the result back into register `A`.

| # | Instruction | Step 3 | Step 4 |
|---|---|---|---|
| 8-11 | `ADD/AND/OR/XOR` | `B_OUT, ACC_WRITE, FLAGS_WRITE` | `ACC_OUT, A_WRITE` |

That is why on step 3 we only have `B_OUT` and no `A_OUT` since `A` is directly connected to the `ALU` already.

But what is `ACC` you might be asking? Well `ACC` is kind of like a temporary register that the `ALU` uses to store its result, the `ALU` can't directly put the result onto the bus, since register `B` is driving the bus.

So on step 3, the `ALU` lets `B` drive the bus, and computes its result storing the flags in the register, and also storing the result. Then in step 4 when `B` stops driving the bus, then we let `ACC` drive the bus and store that value into register `A`.

But now the big question, if all 4 of these instructions run the same micro-steps what makes them different? Have a look at the last 2 bits of the number for each of these instructions in the instruction table. Then have a look at the ALU's operation table.

Also just for future reference, these numbers that represent instructions are called op-codes, just like the `ALU` op-bits.

Yep, the last 2 bits of each instruction directly match the `ALU` bits for that specific operation. Must be a coincidence…

Anyway, because of this, we can just directly wire those 2 last bits into the `ALU` op-bits. Since we only actually store the result of the `ALU` on these instructions, sure a `STORE A, [addr]` instruction might make the `ALU` XOR, but it doesn't matter because we don't store the result, it's garbage.

So here we are:

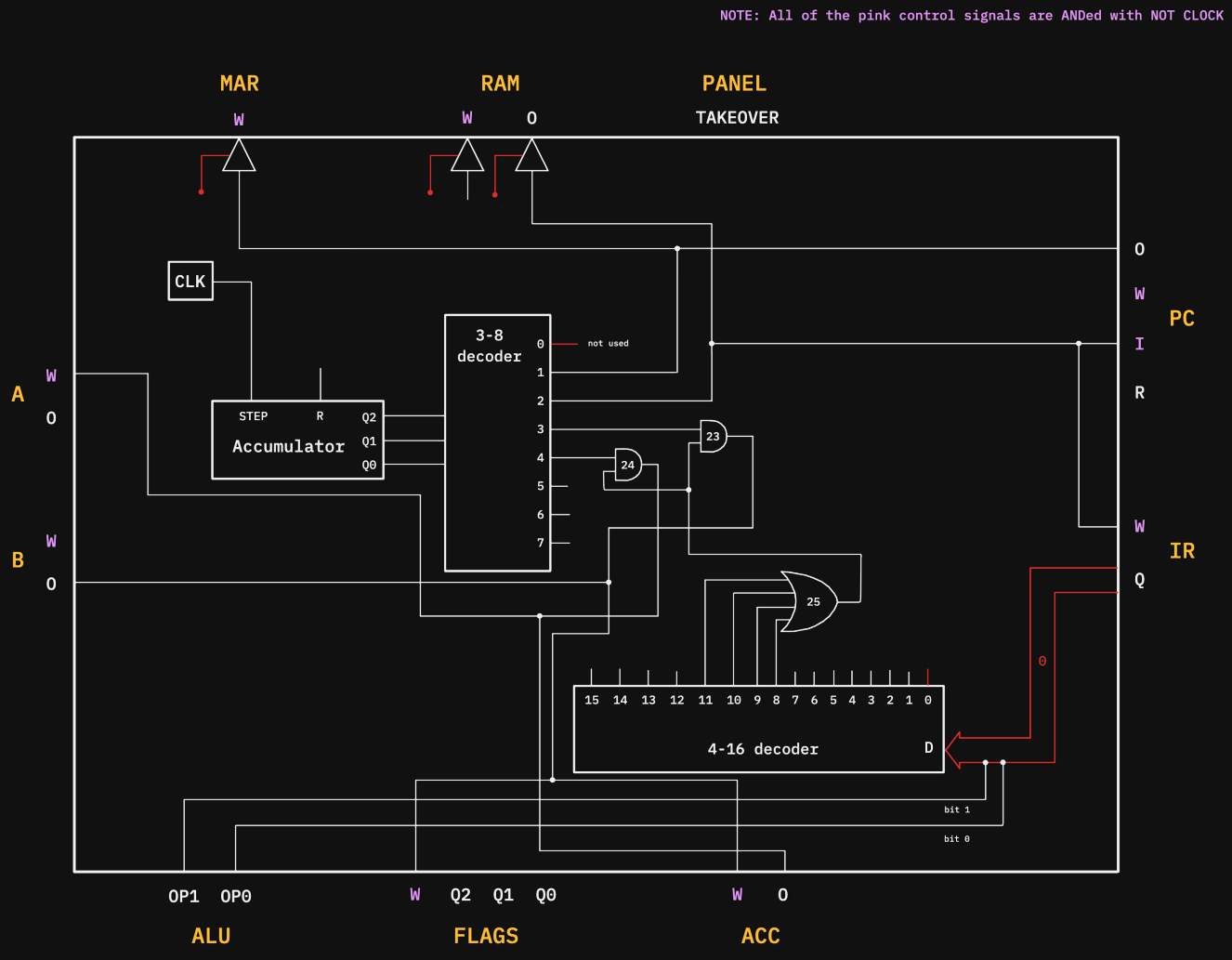



**Diagram 13.7.** The ALU instructions.

Again I am leaving out all the instructions apart from these 4, and again this wiring is a mess.

All you need to know is that, the bottom 2 bits from the instruction op-code are tapped out, and connected directly to the `ALU` ops 0 and 1. And the rest of the logic just checks if it is an `ALU` instruction and if so, on the appropriate steps, it enables the correct control wires according to the table above.

One last thing. The `HALT` instruction, this instruction just stops the clock. The way we do this is by using an SR latch, whose `Q` is inverted and then ANDed with the clock signal itself. So, on step 3 of instruction number 14, we just set that latch, and boom! We froze the computer.

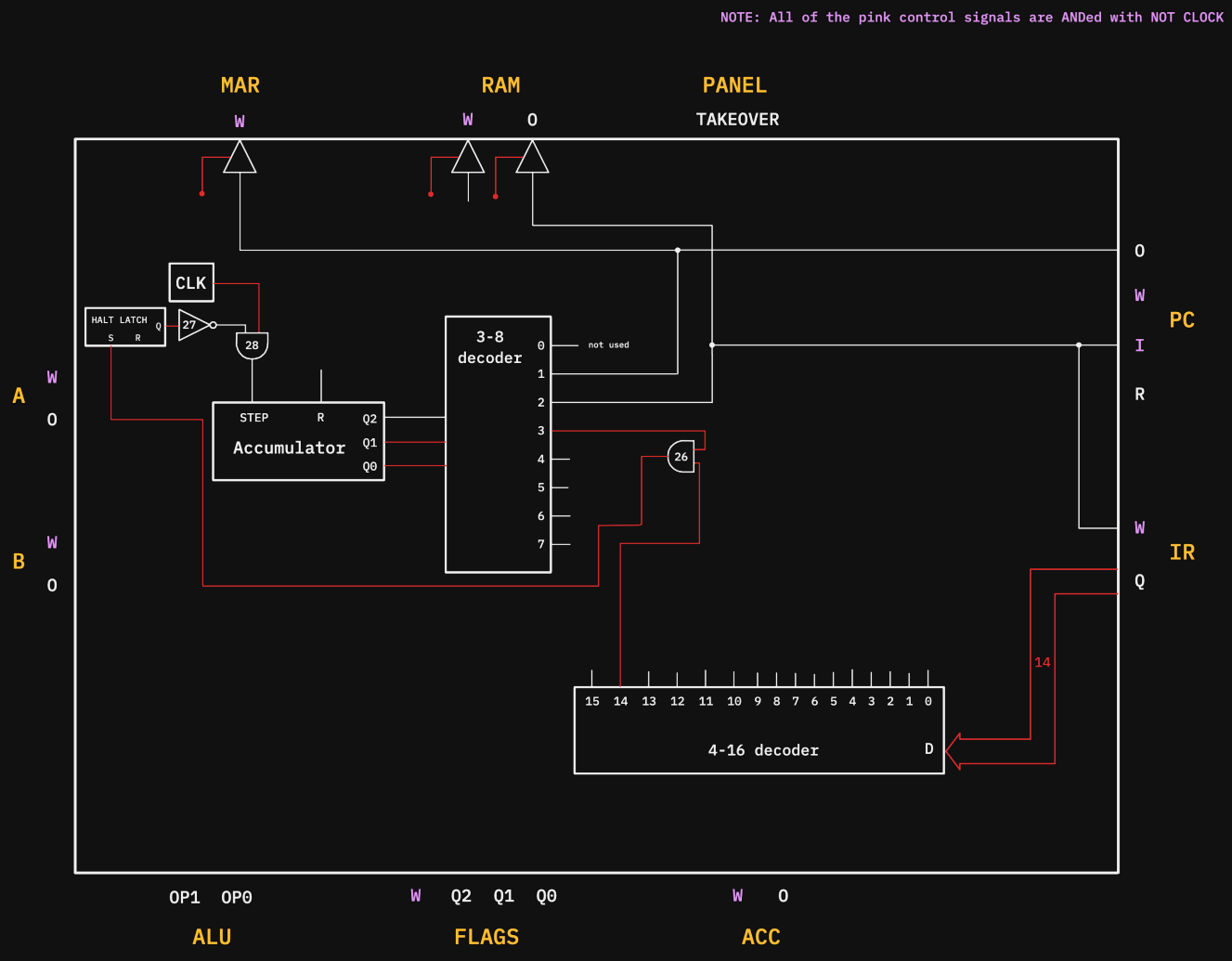



**Diagram 13.8.** The `HALT` instruction freezing the computer.

Even though the clock is on, the computer stays frozen.

But how do we restart the computer? Well, remember the `TAKEOVER` switch I was telling you about. When we flick it on, it resets that latch. But its own inverted signal is ANDed in with the clock and the latch too, so a 3-input AND gate basically.

So the clock stays frozen, but if you flick `TAKEOVER` off, then it will continue going.

TAKEOVER is also connected to PC_RESET which resets the program counter to 0, so the computer will start executing from the first instruction when TAKEOVER is turned off. It also resets the step counter, moving it back to micro-step 0 in preparation for when the computer turns back on.

TAKEOVER also disables the CU's tri-state buffers and enables the panel's tri-state buffers for MAR_WRITE, RAM_WRITE, and RAM_OUT, so the panel can control those signals.

Here is how the HALT and TAKEOVER wiring would look in the CU:

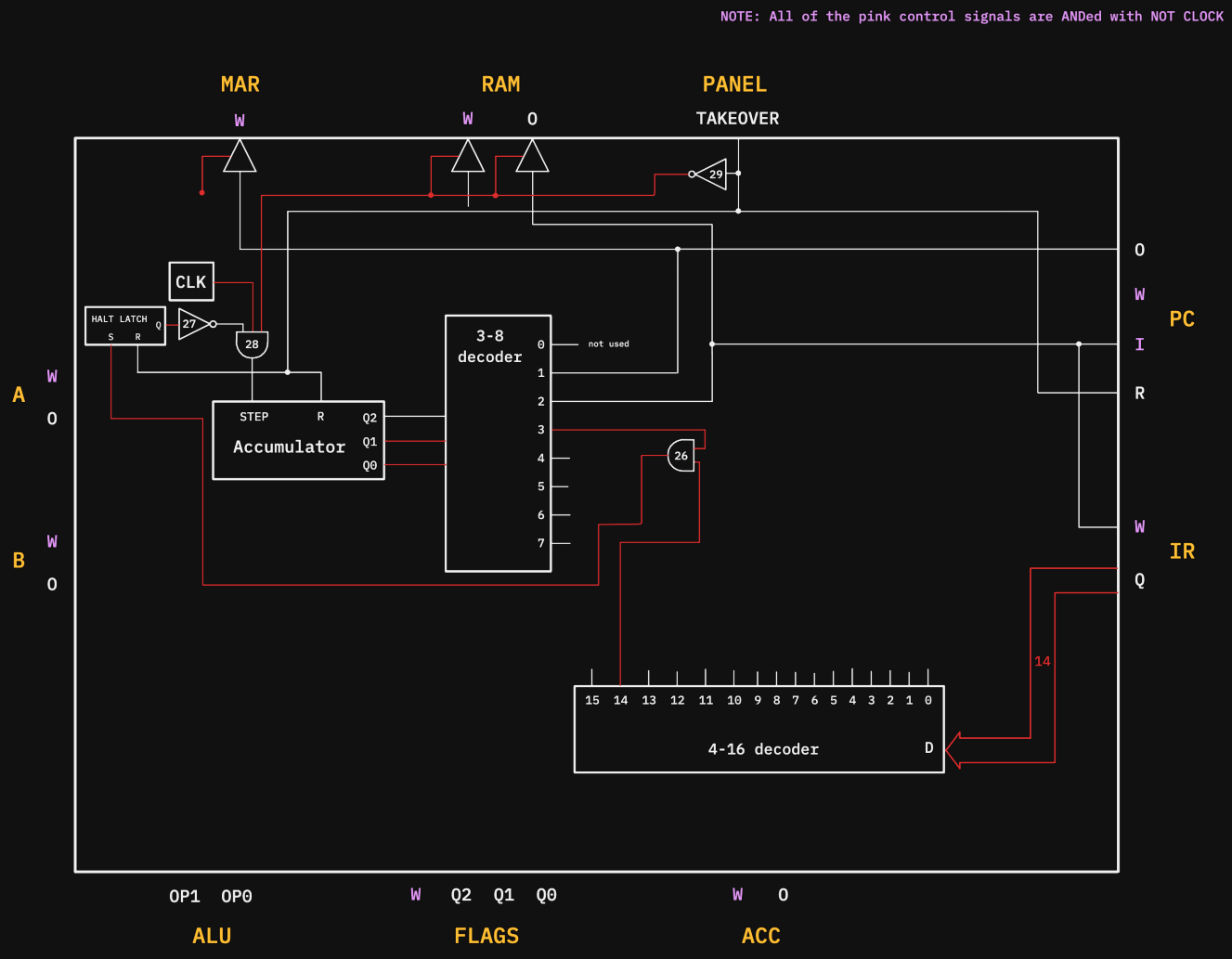



**Diagram 13.9.** The HALT and TAKEOVER wiring.

Wait, is that the whole CU?

Umm, yes, that's it. Just combine all of those diagrams together, add in some extra OR gates and boom. That is the whole CU. You understand it, every wire of it.

In the next short section, let's see how our computer can run a simple program.

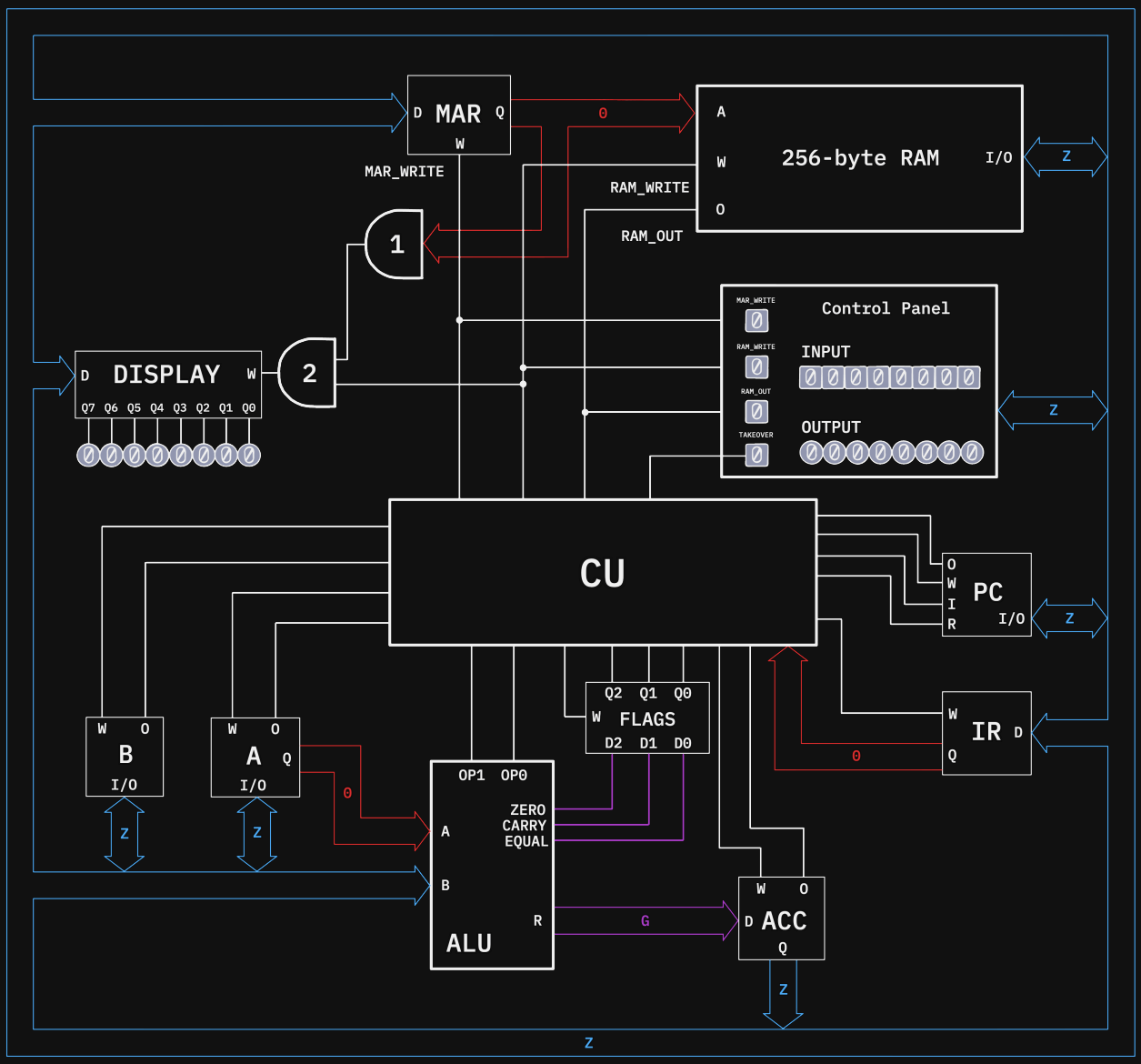
## 3.3. Finally, The Fun Part

That's it huh? A whole computer. To test it, let's watch it run.

Let's start with the simplest program imaginable, just putting a number on the `DISPLAY` lamps:

```
0: LOAD A, 42
2: STORE A, [255]
4: HALT
```

Here is our CPU executing it:




Full 13-frame animation at bytecolony.computer/writing/the-cpu/#f=program-demo:1.

**Diagram 14.1.** The CPU running a simple program.

For this animation, I am not animating the idle steps where nothing happens, and I am not animating 2 frames per micro-step. That's why you might see an `OUT` and a `WRITE` turn on at the same time, but you and me both know what is really happening.

And there it is, `42` sitting on the lamps. Now let's try something real, a multiplication program. Put one number in slot 100, the other in slot 101, and the product ends up on the `DISPLAY` lamps.

```
0:  LOAD A, 0
2:  STORE A, [102]

4:  LOAD A, 0
6:  MOV B, A
7:  LOAD A, [101]
9:  ADD
10: JZ 30

12: LOAD A, [100]
14: MOV B, A
15: LOAD A, [102]
17: ADD
18: STORE A, [102]

20: LOAD A, 255
22: MOV B, A
23: LOAD A, [101]
25: ADD
26: STORE A, [101]

28: JMP 4

30: LOAD A, [102]
32: STORE A, [255]
34: HALT
```

The running total lives in slot `102`. The program adds the value in slot `100` into slot `102`, and then slot `101` counts down. You might notice something weird:

```
20: LOAD A, 255
22: MOV B, A
23: LOAD A, [101]
25: ADD
26: STORE A, [101]
```

This little sequence adds 255 to slot `101`. It loads the right values into the CPU's registers, adds 255, and stores that back into slot `101`. But why?

Well on this computer adding 255 is just like subtracting 1! This is because 255 wraps to $-1$. For example, let's say we have the number 5, we add 255, we get 260, but a byte runs out after 255 and rolls back over to 0, so 260 wraps around to 4!

One more oddity:

```
4:  LOAD A, 0
6:  MOV B, A
7:  LOAD A, [101]
9:  ADD
10: JZ 30
```

Our goal with this chunk of code is to just check if slot `101` is zero, if so we are done and can jump to some other part of the program. Because this CPU is slightly more primitive compared to real CPUs, the programmer (me) had to use some weird trickery to check if it is zero. In this case I just added 0 to the value to set the `ZERO` flag.

This is basically all the program is:

```
total = 0                      (slot 102)
while counter is not 0:        (counter is slot 101)
    total = total + number     (number is slot 100)
    counter = counter - 1
show total on the lamps
halt
```

My point with showing you all of these tricks is that, even though this CPU might not be the most advanced, it can do real things!

Modern CPUs have proper solutions to things like subtraction, negative numbers, division, more flags, and a lot more that smart computer science people just thought of and you can go learn about them yourself. But here is the thing: those dedicated circuits exist mostly for speed. Our funky little CPU can already do all of it, just slower, with tricks like the ones above.

Modern CPUs are also just bigger in general. Instead of 8 wires in a bus, 64. Instead of 256 bytes of `RAM`, billions. Instead of 2 registers, dozens. Instead of a few ticks per second, billions of those too. But it is the same core loop, fetch, decode, execute, just bigger, and faster.

Our program could also be seen as just a bunch of numbers:

```
1 0 3 102 1 0 4 2 101 8 13 30 2 100 4 2 102 8 3 102
1 255 4 2 101 8 3 101 6 4 2 102 3 255 14
```

And in `RAM`, it is actually just this:

```
0:  00000001    12: 00000010    24: 01100101
1:  00000000    13: 01100100    25: 00001000
2:  00000011    14: 00000100    26: 00000011
3:  01100110    15: 00000010    27: 01100101
4:  00000001    16: 01100110    28: 00000110
5:  00000000    17: 00001000    29: 00000100
6:  00000100    18: 00000011    30: 00000010
7:  00000010    19: 01100110    31: 01100110
8:  01100101    20: 00000001    32: 00000011
9:  00001000    21: 11111111    33: 11111111
10: 00001101    22: 00000100    34: 00001110
11: 00011110    23: 00000010
```

Some user would have to manually sit there at the control panel flipping these values into addresses of `RAM` one at a time. In the next closing section, we will touch a little bit on modern things like keyboards and monitors, all that other stuff that really turns a CPU into a computer.

## 3.4. The End

---

So what about keyboards, monitors? All the things that turn a CPU into a computer. Well, you have already seen the core idea. Imagine the `DISPLAY` register, just much bigger: thousands of memory slots instead of one, where each number sets the color of one tiny pixel on a screen. A keyboard is the same thing but in reverse, a slot you read instead of write, where the value is a code for the last key pressed.

Real computers are a lot fancier about it. They have dedicated I/O buses, whole controller chips that talk to the devices so the CPU doesn't have to, and devices that can even write into `RAM` directly on their own.

So to recap, a program is just a list of numbers sitting in `RAM`. The control unit fetches those numbers one at a time, decodes them, and executes each one as a few micro-steps, flipping control wires that move bytes between registers. Every one of those components is made up of simple logic gates, and every gate is made up of relays, or more commonly these days, transistors.

I hope you now understand why computers aren't "just" 1s and 0s, but a very tall pile of simple.